PFDP

Mete Minbay (and Claude)

August 19, 2026

1 Problem definition

1.1 The objective

Fix the suffix array I of the text, and let $m = |I|$ denote the number of rows in I. For a word w, let $I_w = [\ell_w, r_w)$ denote the maximal LCP-interval in I such that every suffix in I_w starts with w. At a high level, we would like to prune I by finding contiguous rows whose positions in the text can be recreated by adding a constant offset to the positions of a different contiguous window (in which case the former row range can be deleted and replaced with a constant-size hash entry denoting "this row range is deleted; fetch the positions of this other range and add this offset instead").

Towards this goal, a **candidate** is a triple $k = (T_k, S_k, a_k)$: a contiguous target interval $T_k \subseteq I_w$ of some word w, a contiguous source interval $S_k \subseteq I_{w'}$ of another word w', and a shift $a_k \in [a_{max}]$ such that the a_k-th suffix in the suffices in $S_{w'}$ reconstruct the suffices in T_w in order. In terms of compression, accepting a candidate k reduces m by deleting the rows T_k and instead reconstructing their positions via adding a shift a_k to every position in S_k. Let U denote the universe of all candidates, and $\text{cov}(k) = |T_k|$ denote the number of rows that are to be deleted when a candidate k is accepted. Under a subset of accepted candidates $X \subseteq U$, the resulting size of I becomes

$$m - \sum_{k \in X} \text{cov}(k)$$

Assuming that the space usage of our k-mer lookup structure increases monotonically with the number of remaining rows [1] , the objective is to find a subset of consistent candidates that minimize the above expression.

1.2 The constraints

Accepted candidates are subject to the following constraints:

- **One reference per word**: After the suffix array is compressed, constant-time lookup of words is achieved with a hash table that stores the *real* and the *referenced* occurrences of a word. Every word can have at most one entry of each kind. Therefore, candidates k and k' cannot be accepted simultaneously if $T_k, T_{k'} \subseteq I_w$ for some w.
- **Source-integrity**: If a row range T_k has been deleted to reference another range S_k, the source range S_k cannot be deleted. Therefore, candidates k and k' cannot be accepted simultaneously if $S_k \cap T_{k'} \neq \emptyset$.

[1] This assumption is not completely accurate: Since every accepted candidate increases the number of hash table entries, accepting many small-coverage candidates can cause the HT index to take up extra space that outweigh the space savings of the accepted candidates.

1.3 Maximum weighted independent set

Consider a graph G where the vertices are the candidates and two candidates are connected if they cannot be accepted together due to the listed constraints, formally $V(G) = U$ and $E(G) = \{(k, k') : \exists w(T_k, T_{k'} \subseteq I_w) \vee S_k \cap T_{k'} \neq \emptyset\}$. Let the weight of each vertex be $w(k) := \text{cov}(k)$. Maximizing the number of deleted rows is equivalent to finding a maximum weighted independent set (MWIS) of G.

2 The method

2.1 Overview

Finding a MWIS admits a polynomial time dynamic programming algorithm on trees, but is NP-hard on general graphs. It is easy to see that the conflict graphs of our instances are not trees: Since every word can have ≤ 1 accepted candidate, multiple candidates belonging to the same word are already modeled as a clique. Solving the problem exactly is therefore unlikely unless the LCP-intervals give the graph some structure to be exploited.

The main idea of PFDP is to heuristically sparsify the candidate space, then exactly solve the problem on the remaining candidates using dynamic programming.

2.2 Prerequisite: MWIS on trees

The MWIS of a tree $G = (V, E)$ can be computed exactly with dynamic programming as follows. Let $r \in V$ be the root of G, and for any vertex $v \in V$ let $C(v) \subseteq V$ denote the children of v. Define $A[v]$ as the MWIS of the *subtree rooted at* v where v is fixed to be part of the independent set. Define $B[v]$ analogously but fix v not to be part of the independent set. The MWIS is then $\max\{A[r], B[r]\}$. The recursion carries out as follows:

$$A[v] = w(v) + \sum_{u \in C(v)} B[u]$$

$$B[v] = 0 + \sum_{u \in C(v)} \max\{A[u], B[u]\}$$

(Intuitively, fixing v to be part of the solution forces the children not to be part of the solution. Fixing v not to be in the solution has no constraints on the children. Notice the summation term does not exist for leaves so they serve as base cases)

There are $2n$ subproblems to solve, and each subproblem compares sums up many cases as the number of neighbors of the associated vertex. This already bounds the running time at $\mathcal{O}(n^2)$, but notice that every non-root vertex only appears as a summand once. Thus the total number of additions is also bounded linearly and the total running time is $\mathcal{O}(n)$.

2.3 Sparsifying the graph

For some word w, define its candidates $K_w = \{k : T_k \subseteq I_w\}$ as the candidates that propose deleting rows of I_w. We define the *preferred* candidate $k_w \in K_w$ to be the one with the highest coverage (ties broken in favor of larger a_k) [2]. Let $U' \subseteq U$ be the universe restricted to preferred candidates, and consider the arising conflict graph G'.

There are no conflicts between candidates targeting the same word, because the graph has been restricted to a single candidate targeting each word. So the only conflicts on this graph are arising from source-target intersections. Consider directing the edges of the graph such that there is an edge $k \to k'$ if the source of k intersects the target of k'. Under this orientation,

[2] The preference rule is a heuristic aiming to delete the largest number of rows per word. Different rules may provide better results.

every vertex has at most one outgoing edge: Any two candidates whose targets intersect with the source of k must be targeting the same word (due to the disjointness of LCP intervals), but there can only be candidate targeting each word. The resulting graph is therefore a *pseudoforest*: Every component with t vertices has exactly t edges.

2.4 MWIS on pseudoforests

A pseudoforest consists of *pseudotrees*, which are trees with an additional edge (that creates exactly one cycle). The MWIS algorithm for trees can be adapted for pseudoforests by fixing a cycle node and branching on its inclusion in the independent set. Specifically, let P be a pseudoforest and $v \in V(P)$ be a cycle node, and let $N[v]$ denote the neighbors of v. There are two cases:

- If v is in the solution: $\text{MWIS}(P) = w(v) + \text{MWIS}(P - (\{v\} \cup N[v]))$
- If v is not in the solution: $\text{MWIS}(P) = 0 + \text{MWIS}(P - \{v\})$.

Notice that removing v already destroys the cycle and leaves behind a forest, so the smaller subproblems of either case can be solved exactly in $\mathcal{O}(n)$ time. The overall running time is therefore $\mathcal{O}(n)$.

2.5 Leftover phase

After one pass of the DP algorithm on the preferred candidates, some words end up with no accepted candidates because their preferred candidate was not part of the DP solution. Such words may have had other (non-preferred) candidates that do not conflict with the DP solution, but these candidates were eliminated in the sparsification phase and therefore were not considered yet. These candidates are accounted for with a singular greedy pass. After the DP solution is finalized, all candidates are revisited in decreasing order of coverage and added to the solution if they do not conflict with what is already in the solution.

2.6 IDEA: Iterative DP

Instead of applying the DP once on preferred candidates and immediately applying the greedy pass on leftover candidates, we can apply the DP iteratively. After the first application, we fix the set of accepted candidates and extract a second set of preferences that do not conflict with the fixed solution. We can then apply DP on this second set and add the new solution to the fixed solution, and repeat this process until no more candidates can be extracted.